

Building "ANSA Flasher"

A Raspberry Pi Imager-style install tool for the ANSA STM32 firmware

0. The Starting Point: How It Differs From Raspberry Pi Imager

Raspberry Pi Imager writes a **raw disk image** to an SD card — the Pi's OS lives entirely on removable storage, making installation a block-level copy. An STM32, however, has no removable storage. Its firmware resides in the chip's internal flash, requiring a **programming/debug interface** rather than a disk write.

Therefore, "ANSA Flasher" solves a different problem with the same UX: instead of writing bytes to a block device, it communicates via a specific protocol to erase and reprogram internal flash. This is standard for consumer IoT devices, allowing the UX to mimic Raspberry Pi Imager despite the underlying differences.

1. Three Ways to Flash Firmware onto an STM32 over USB

You must select a transport method based on your board's physical wiring.

1.1 ST-Link (SWD) — The Nucleo Standard

Nucleo boards (like the F446RE) feature a secondary MCU acting as a USB-to-SWD bridge (the "ST-Link"). The host sees the ST-Link, which then uses SWD (Serial Wire Debug) to communicate with the target STM32.

- **Pro:** Reliable, standard on dev boards, supports live debugging (breakpoints, memory inspection).
- **Con:** Requires an onboard ST-Link or an external dongle. Production boards rarely include this hardware.
- **Host Tool:** `STM32_Programmer_CLI` or open-source `openocd` / `st-flash`.

1.2 USB DFU — The Production Standard

Every STM32 includes a ROM bootloader that supports **USB DFU** (Device Firmware Upgrade) if the chip's USB peripheral is wired to a connector and the `BOOT0` pin is pulled high at reset.

- **Pro:** No extra hardware required. Ideal for end-user flashing (e.g., Black Pill STM32F411CEU6).
- **Con:** Depends on specific chip ROM support and requires physical access to `BOOT0` (via button or firmware-controlled GPIO).
- **Host Tool:** `dfu-util` (open source, cross-platform).

1.3 UART Bootloader — The Fallback

The ROM bootloader also supports UART (ST AN3155 protocol) if a USB-to-serial adapter is wired correctly. It is slower and requires extra hardware but serves as a solid fallback.

- **Host Tool:** `stm32flash` (open source).

Recommendation: Support DFU as the primary method for a "plug and play" experience, and SWD as secondary for development. Keep UART as a documented fallback.

2. Application Architecture

2.1 Framework Choice

Framework	Description	Pros/Cons
Qt (C++/QML)	Used by Raspberry Pi Imager.	Native look, small binary, but has a steep learning curve and C++ toolchain overhead.
Electron	Chromium + Node.js.	Fast UI development (web tech), but results in large binaries (100MB+) and high memory usage.
Tauri	Rust backend + system webview.	Modern middle ground: web-tech UI, small binary, excellent for low-level USB/serial backend code.

Recommendation: Tauri. It offers visual polish without Electron's bloat, and Rust is ideal for hardware communication.

2.2 Backend Requirements

1. **Fetch Manifest:** Simple HTTP GET and JSON parsing.
2. **Enumerate USB Devices:** Match against a known board database by VID:PID.
 - ST-Link: VID `0483` , PID `3748` / `374B` / `374E`
 - DFU mode: VID `0483` , PID `DF11`
 - Use `libusb` (via `rusb`) for cross-platform support.
3. **Download & Verify:** Download firmware and verify checksum/signature before flashing.
4. **Invoke Flasher:** Shell out to bundled `dfu-util` / `STM32_Programmer_CLI` or link against ST's DFU implementation.

Recommendation: For version 1, shell out to a bundled `dfu-util` binary. It is mature and cross-platform.

3. The Manifest Structure

A JSON manifest defines available firmware, similar to Raspberry Pi Imager. Example:

```
{
  "boards": [
    {
      "id": "ansa-f446re",
      "name": "ANSA OS - Nucleo-F446RE",
      "mcu": "STM32F446RE",
      "transport": "swd",
      "versions": [
        {
          "version": "1.2.0",
          "channel": "stable",
          "url": "https://ansa.dev/firmware/f446re/ansa-1.2.0.bin",
          "sha256": "a1b2c3...",
          "signature_url": "https://ansa.dev/firmware/f446re/ansa-1.2.0.bin.sig",
          "size_bytes": 131072
        }
      ]
    }
  ]
}
```

4. The Install Flow

1. **Board Select:** List supported boards with images.
2. **Version Select:** Choose stable/beta channels.
3. **Customize (Optional):** Set drone ID, WiFi config, etc., written to a reserved flash page.
4. **Connect Device:** Auto-detect matching VID:PID without requiring user interaction.
5. **Confirm:** Display board info and warning before erasure.
6. **Flashing:** Show progress bar parsing `dfu-util` output.
7. **Done:** Success screen. Optionally, listen on UART for the boot banner to confirm successful boot.

5. Security

Firmware distribution requires robust security:

- **SHA-256 Checksum:** Catches download corruption.

- **Cryptographic Signature:** Sign binaries with a private key; embed the public key in the app to prevent flashing tampered firmware.
- **Code-Sign App:** Use Authenticode (Windows) and Notarization (macOS) to avoid OS warnings.

6. Packaging & OS Integration

Tauri handles packaging for Windows (`.msi` / `.exe`), macOS (`.dmg`), and Linux (`.AppImage` / `.deb`). Include an auto-update mechanism so users always have the latest manifest schema.

7. Firmware Polish

Make the firmware feel like a proper OS:

- **Boot Banner:** Print an ASCII banner (version, build, board) over UART at startup.
- **Version Command:** Add an `about` command to the CLI.
- **Status LEDs:** Implement consistent blink patterns for boot success or faults.

8. Suggested Build Order

1. Bundle `dfu-util` + `STM32_Programmer_CLI` and write a CLI wrapper.
2. Create and host the manifest JSON.
3. Build the Tauri UI shell (board picker, device polling).
4. Integrate the flash wrapper into the UI with progress tracking.
5. Implement signature verification.
6. Add the boot banner to the firmware.
7. Package, code-sign, and setup the auto-updater.
8. Create the download webpage with OS auto-detection.